

Registros (struct)

Declaração, acesso, registros aninhados e o layout dos dados na memória

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

andreyoshiaki@dcomp.ufs.br

Um programa para começar

Problema:

Um aluno tem nome, matrícula, data de nascimento e média. Com o que sabemos até aqui, guardamos isso em **vetores paralelos**: um para nomes, outro para matrículas, outro para médias.

Nada garante que `nome[3]` e `media[3]` sejam da *mesma* pessoa. Ordenar por média exige reordenar todos os vetores em sincronia.

Solução:

Um **registro** agrupa campos de tipos diferentes sob um único nome. O aluno passa a ser *um* valor: cabe em uma variável, entra em um vetor, vira parâmetro e vira retorno de função.

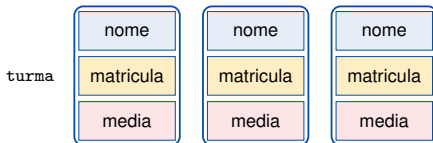
“Os dados de um aluno” deixam de ser convenção na cabeça do programador e viram um **tipo** que o compilador conhece.

Vetores paralelos

nome	Ana	Bia	Caio
matricula	202600123	202600124	202600125
media	8.75	6.50	9.10

sincronia mantida “na mão”:
trocar um vetor exige trocar os outros

Vetor de registros



cada aluno é *um* valor:
`turma[0]`, `turma[1]`, `turma[2]`

Um programa para começar: a ficha do aluno



```
1  #include <stdio.h>
2  #include <string.h>
3
4  struct Data {
5      int dia;
6      int mes;
7      int ano;
8  };
9  struct Aluno {
10     char    nome[20];
11     int     matricula;
12     struct Data nascimento;
13     double  media;
14  };
```

```
15  int main(void) {
16     struct Aluno a;
17     strcpy(a.nome, "Ana Souza");
18     a.matricula = 202600123;
19     a.nascimento.dia = 7;
20     a.nascimento.mes = 3;
21     a.nascimento.ano = 2005;
22     a.media = 8.75;
23     printf("%s (%d)\n", a.nome, a.
24           matricula);
25     printf("Media: %.2f\n", a.media);
26     printf("sizeof(Aluno) = %zu\n",
27           sizeof(struct Aluno));
28     return 0;
29 }
```

Em duplas, anotem a previsão (3–4 min) — sem computador!

1. Quais são exatamente as três linhas impressas na tela?
2. Quantos *bytes* ocupa `struct Data`? E `struct Aluno`?
3. Some o tamanho de cada campo de `struct Aluno` separadamente. Esse total tem que bater com `sizeof(struct Aluno)`?

Regra do jogo

Escreva um número, não um “mais ou menos”. Previsão errada é o ponto de partida da investigação — e a pergunta 3 é a mais interessante da aula.

```
$ gcc ficha_aluno.c -o ficha_aluno
$ ./ficha_aluno
Ana Souza (202600123)
Media: 8.75
sizeof(Aluno) = 48
```

A soma dos campos não fecha

`char nome[20] = 20, int matricula = 4, struct Data = 12, double media = 8.`
Total: **44 bytes**. Mas `sizeof` respondeu **48**.

A pergunta que guia o resto da aula

Onde estão os 4 bytes que ninguém pediu — e quem decidiu colocá-los ali?

Registros: declaração e acesso

Definição

Um **registro** (`struct`) é um tipo composto que agrupa um número fixo de **campos** (ou membros), possivelmente de tipos diferentes, sob um único nome. Cada campo tem nome próprio e ocupa uma região distinta da memória do registro.

Intuição

Um vetor é uma *fila de itens iguais* acessados por índice; um registro é uma *ficha de campos diferentes* acessados por nome. O vetor responde “qual é o terceiro?”; o registro responde “qual é a média?”.

```
1  struct Aluno {           /* Aluno e' a TAG do registro */
2      char   nome[20];     /* campo 1                */
3      int    matricula;    /* campo 2                */
4      double media;        /* campo 3                */
5  };                       /* ; obrigatorio aqui!   */
6
7  struct Aluno a;          /* declara UMA variavel do tipo */
8  struct Aluno turma[40]; /* vetor de 40 registros        */
```

- A declaração cria um **tipo**, não uma variável: nenhuma memória é reservada até você escrever `struct Aluno a;`.
- O nome completo do tipo é `struct Aluno` — as duas palavras.
- Esquecer o `;` depois do `}` é o erro mais comum.

```
1  /* sem typedef */
2  struct Aluno {
3      char nome[20];
4      int  matricula;
5  };
6
7  struct Aluno a;
8  struct Aluno turma[40];
```

```
1  /* com typedef */
2  typedef struct {
3      char nome[20];
4      int  matricula;
5  } Aluno;
6
7  Aluno a;
8  Aluno turma[40];
```

Qual usar?

As duas formas geram exatamente o mesmo tipo em memória. O typedef encurta o código; a forma com struct deixa explícito que o tipo é um registro. Nesta aula usaremos struct Nome para não esconder nada.

```
1 struct Aluno a;  
2  
3 a.matricula = 202600123;      /* escrita: campo a esquerda do = */  
4 a.media      = 8.75;  
5 strcpy(a.nome, "Ana Souza"); /* vetor de char: use strcpy      */  
6  
7 printf("%d\n", a.matricula); /* leitura                      */  
8 a.media = a.media + 0.5;     /* campo se usa como variavel */
```

- `a.media` é uma variável `double`: vale em toda expressão onde um `double` valeria.
- `a.nome` é um vetor — `a.nome = "Ana"` **não** compila; use `strcpy`.

```
1  /* 1. por lista, na ordem de declaracao dos campos */
2  struct Aluno a = {"Ana Souza", 202600123, 8.75};
3  /* 2. campos nomeados (C99) -- ordem livre, resto zerado */
4  struct Aluno b = {.matricula = 202600124, .media = 7.0};
5  /* 3. tudo zerado */
6  struct Aluno c = {0};
7  /* 4. campo a campo, depois da declaracao */
8  struct Aluno d;
9  d.matricula = 202600125;
```

Cuidado

Sem inicialização, os campos de `d` contêm **lixo** — o que estava naquela região da pilha. Ler antes de escrever dá valores imprevisíveis.

```
1 struct Aluno a = {"Ana Souza", 202600123, 8.75};
2 struct Aluno b;
3
4 b = a;           /* copia OS 48 BYTES: b e a sao independentes */
5 b.media = 10.0;  /* nao afeta a.media */
6
7 int v[3] = {1,2,3}, w[3];
8 w = v;           /* ERRO de compilacao: vetor nao se atribui */
```

Consequência prática

O registro é um **valor**: pode ser atribuído, passado por cópia e devolvido por return. Repare que o vetor `nome[20]` dentro do registro também é copiado — “vetor não se atribui” vale para o vetor solto, não para o campo.

```
1  /* recebe uma COPIA: alteracoes nao saem da funcao */
2  void imprime(struct Aluno x) {
3      printf("%-20s %d %.2f\n", x.nome, x.matricula, x.media);
4  }
5
6  /* devolve um registro construido dentro da funcao */
7  struct Aluno cria(const char *nome, int mat, double media) {
8      struct Aluno novo;
9      strcpy(novo.nome, nome);
10     novo.matricula = mat;
11     novo.media     = media;
12     return novo;    /* copia de volta para quem chamou */
13 }
```

Guarde esta pergunta

E se o registro tivesse 500 bytes e fosse passado assim dentro de um laço de um milhão de iterações? A resposta — passar o *endereço* — é a próxima aula.

```
1 struct Aluno turma[3] = {
2     {"Ana Souza", 202600123, 8.75},
3     {"Bia Rocha", 202600124, 6.50},
4     {"Caio Lima", 202600125, 9.10}
5 };
6
7 double soma = 0.0;
8 for (int i = 0; i < 3; i++)
9     soma += turma[i].media;    /* indexa o vetor, DEPOIS pega o campo */
10
11 printf("Media da turma: %.2f\n", soma / 3);
```

- `turma[i].media`: primeiro `[i]` escolhe o registro, depois `.media` escolhe o campo dentro dele.
- Ordenar a turma por média agora é trocar registros inteiros — e a ficha de cada aluno viaja junta, sem risco de dessincronizar.

O que aprendemos

- `struct Tag { campos };` cria um **tipo**; a memória só aparece ao declarar uma variável.
- Campos se acessam por nome com o operador ponto e valem como variáveis comuns.
- Inicialização: por lista, por campos nomeados, com `{0}` ou campo a campo — sem inicializar, há lixo.
- Registros se atribuem, se passam para funções e se retornam — sempre por **cópia**.

Registros aninhados

Problema:

A data de nascimento tem três partes que só fazem sentido juntas.

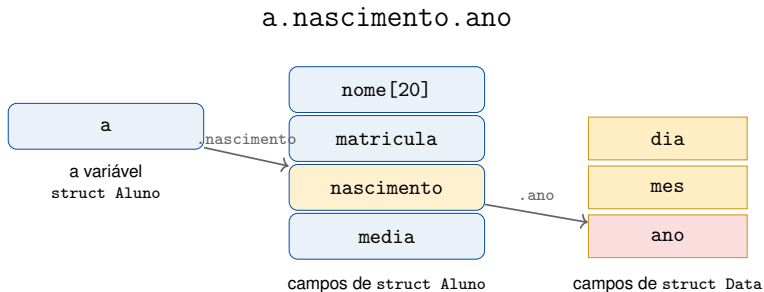
Espalhá-las como `dia_nasc`, `mes_nasc`, `ano_nasc` dentro de `struct Aluno` funciona — até aparecer a data de matrícula, e depois a de colação. Aí são nove campos soltos e nenhuma função que receba “uma data”.

Solução:

Declarar `struct Data` uma vez e **usá-la como tipo de campo**. Toda função que valida, compara ou imprime datas passa a servir a qualquer registro que contenha uma — e o conceito “data” fica escrito em um lugar só.

```
1  struct Data {                      /* precisa vir ANTES de ser usada */
2      int dia, mes, ano;
3  };
4
5  struct Aluno {
6      char    nome[20];
7      int     matricula;
8      struct Data nascimento;        /* campo do tipo struct Data */
9      struct Data matriculado;       /* outro campo, mesmo tipo    */
10     double   media;
11 };
```

- struct Data tem de estar declarada antes — o compilador precisa saber o tamanho do campo.
- Um registro **não** pode conter a si mesmo (seria de tamanho infinito); só um *ponteiro* para si mesmo.



Regra

Cada ponto desce um nível. `a.nascimento` é um valor completo do tipo `struct Data`; `a.nascimento.ano` é o `int` dentro dele.

```
1  struct Data hoje = {17, 8, 2026};
2
3  struct Aluno a;
4  a.nascimento = hoje;           /* copia a struct Data inteira */
5  a.nascimento.ano = 2005;       /* ajusta so um campo interno */
6
7  /* funcao que serve a QUALQUER data, venha de onde vier */
8  int idade_em(struct Data nasc, struct Data ref) {
9      int anos = ref.ano - nasc.ano;
10     if (ref.mes < nasc.mes || (ref.mes == nasc.mes && ref.dia < nasc.dia))
11         anos--;
12     return anos;
13 }
14
15 printf("%d anos\n", idade_em(a.nascimento, hoje)); /* 21 anos */
```

Repare

a.nascimento entrou na função como argumento: o campo interno é um valor completo.

```
1  /* chaves internas espelham o aninhamento */
2  struct Aluno a = {"Ana Souza", 202600123, {7, 3, 2005}, 8.75};
3
4  /* com campos nomeados, fica auto-explicativo */
5  struct Aluno b = {
6      .nome      = "Bia Rocha",
7      .matricula = 202600124,
8      .nascimento = {.dia = 12, .mes = 11, .ano = 2004},
9      .media     = 6.5
10 };
```

- As chaves internas são opcionais para o compilador, mas obrigatórias para quem vai ler o código depois.
- Em vetores: turma[i].nascimento.ano — índice primeiro, depois os pontos.

O que aprendemos

- Um registro pode ser campo de outro, desde que declarado antes.
- Cada ponto desce um nível: `a.nascimento.ano`.
- O campo interno é um valor completo: pode ser atribuído de uma vez e passado para funções.
- Aninhar concentra um conceito (“data”) em um lugar só e permite reutilizar as funções que operam sobre ele.

Alinhamento e padding na memória

O que contamos:

Campo	Bytes
char nome[20]	20
int matricula	4
struct Data nascimento	12
double media	8
Soma	44

O que o compilador respondeu:

```
sizeof(struct Aluno) = 48
```

A diferença não é bug: o compilador **inseriu bytes** de propósito, para que cada campo comece em um endereço que o processador lê de uma vez.

Vocabulário de hoje

Alinhamento — em que endereços um tipo pode começar. **Offset** — a que distância do início da struct o campo começa. **Padding** — bytes vazios inseridos para o offset dar certo.

int **alinhado** (offset 4)



1 leitura de palavra

int **desalinhado** (offset 2)



2 leituras + junção

As barras douradas marcam as fronteiras de palavra da memória (4 bytes).

A regra do hardware

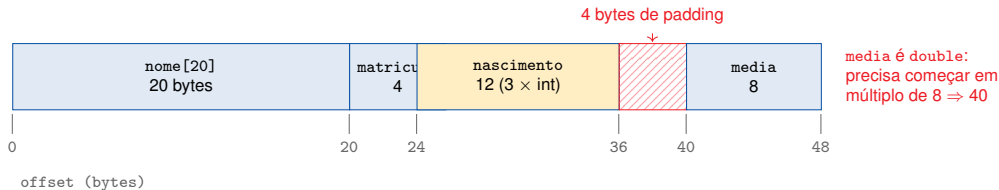
Um tipo de N bytes quer começar em um endereço múltiplo de N . Em x86 o acesso desalinhado é lento; em várias arquiteturas ARM e RISC ele simplesmente **falha**. O compilador prefere gastar bytes a arriscar isso.

Ao montar o registro, na ordem em que você escreveu os campos

1. Cada campo tem um **alinhamento** a — nos tipos básicos, igual ao tamanho (char 1, short 2, int 4, double 8).
2. O campo é colocado no **próximo offset múltiplo de a** ; os bytes pulados viram **padding interno**.
3. O alinhamento da struct é o **maior** entre os de seus campos, e o tamanho total sobe até o próximo múltiplo dele — o **padding final**.

Por que a regra 3 existe

Em `struct Aluno turma[40]`, cada elemento começa `sizeof(struct Aluno)` bytes depois do anterior. Sem múltiplo de 8, o campo `media de turma[1]` cairia desalinhado.



Conferindo

Offsets: nome 0, matricula 20, nascimento 24, media 40. Último byte usado: 47.
Tamanho 48, múltiplo de 8 — aqui não sobrou padding final, só os 4 bytes internos antes de media.

```
1  #include <stdio.h>
2  #include <stddef.h>    /* offsetof */
3
4  int main(void) {
5      printf("sizeof = %zu\n", sizeof(struct Aluno));
6      printf("nome      @ %zu\n", offsetof(struct Aluno, nome));
7      printf("matricula @ %zu\n", offsetof(struct Aluno, matricula));
8      printf("nascimento @ %zu\n", offsetof(struct Aluno, nascimento));
9      printf("media      @ %zu\n", offsetof(struct Aluno, media));
10     return 0;
11 }
```

Saída (Linux/macOS 64 bits)

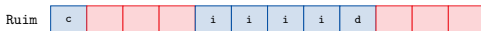
sizeof = 48 nome @ 0 matricula @ 20 nascimento @ 24 media @ 40

Não decore números

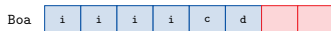
Os tamanhos dependem da arquitetura: aprenda o *mecanismo* e meça.

```
1 struct Ruim {  
2     char c;  
3     int i;  
4     char d;  
5 };  
6 /* sizeof = 12 */
```

```
1 struct Boa {  
2     int i;  
3     char c;  
4     char d;  
5 };  
6 /* sizeof = 8 */
```



6 bytes desperdiçados



2 bytes de padding final

Regra prática

Declare os campos do **maior alinhamento para o menor**. Os mesmos dados ocuparam 33% menos memória — em um milhão de registros, isso são megabytes.

Importa

- Vetores grandes — o padding é multiplicado pelo número de elementos.
- Sistemas embarcados, com RAM escassa.
- Desempenho: mais registros por linha de cache.

Não importa

- Uma ou poucas variáveis do tipo.
- Quando reordenar prejudica a leitura sem ganho medido.

Armadilha clássica

`fwrite(&a, sizeof a, 1, f)` grava *também* o padding, com lixo dentro — outra máquina pode ler tudo errado.

O que aprendemos

- `sizeof` de um registro $\geq$ soma dos campos: a diferença é **padding**.
- Cada campo começa em um offset múltiplo do seu alinhamento; o tamanho total é múltiplo do maior alinhamento da struct.
- O padding existe porque o processador lê a memória em blocos alinhados — é preço de velocidade e de correção.
- Reordenar campos do maior para o menor reduz o desperdício, e `sizeof/offsetof` medem o resultado.

Tarefas

Tarefa 1 — prever antes de compilar

Acrescente `char turma;` a `struct Aluno`, logo depois de `matricula`. Anote sua previsão de `sizeof` *antes* de compilar; depois meça. Explique a diferença (ou a ausência dela).

Tarefa 2 — reordenar

Reordene os campos de `struct Aluno` do maior alinhamento para o menor e meça `sizeof` antes e depois. Deu ganho? Depois repita com `struct { char flag; double valor; int id; char tipo; }` e explique por que os dois casos se comportam de forma diferente.

Tarefa 3 — aninhar mais um nível

Crie `struct Endereco` (rua, número, CEP) e inclua-a em `struct Aluno`. Imprima `a.endereco.cep` e recalcule o tamanho total.

Cadastro da turma

Escreva um programa que:

1. declare `struct Aluno turma[5]` com data aninhada;
2. preencha os cinco alunos no próprio código;
3. imprima uma tabela com nome, matrícula, nascimento e média;
4. informe o aluno de maior média e a média da turma;
5. imprima `sizeof(struct Aluno)`, a soma dos campos e quantos bytes de padding existem por aluno e no vetor inteiro.

Entrega

O tutorial traz o esqueleto, os testes e as perguntas de investigação.

struct agrupa campos de tipos diferentes em um valor único



O ponto acessa campos; cada ponto desce um nível de aninhamento



O compilador insere padding para alinhar cada campo



sizeof e offsetof medem; a ordem dos campos decide o custo

Próxima aula

Ponteiros: em vez de copiar 48 bytes, passar um endereço — e o operador `->` para chegar aos campos a partir dele.



André Backes.

Linguagem C: completa e descomplicada.

Elsevier, Rio de Janeiro, 2013.



Randal E. Bryant and David R. O'Hallaron.

Computer Systems: A Programmer's Perspective.

Pearson, Boston, 3 edition, 2015.

Seção 3.9: alinhamento de dados e layout de structs.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.

Algoritmos: teoria e prática.

Elsevier, Rio de Janeiro, 3 edition, 2012.



Paul Deitel and Harvey Deitel.

C: como programar.

Pearson Prentice Hall, São Paulo, 6 edition, 2011.



Ulrich Drepper.

What Every Programmer Should Know About Memory.

Red Hat, Inc., 2007.

Seção 6.2: efeito da ordem de percurso de matrizes sobre a cache.



Ulrich Drepper.

How to write shared libraries.

Documento técnico, 2011.

Versão 4.1.2.



Paulo Feofiloff.

Algoritmos em linguagem C.

Elsevier, Rio de Janeiro, 2009.



Free Software Foundation.

Gnu coding standards.

Free Software Foundation.

Acesso em 2026.



Free Software Foundation.

Using the GNU compiler collection — options to request or suppress warnings.

Manual do GCC.

Acesso em 2026.



ISO/IEC.

Iso/iec 9899:2018 — information technology: Programming languages: C.

Technical report, International Organization for Standardization, 2018.

Seção 6.7.2.1: structure and union specifiers.



Brian W. Kernighan and Rob Pike.

The Practice of Programming.

Addison-Wesley, Reading, MA, 1999.

Cap. 1, “Style”: nomes, expressões, consistência, números mágicos e comentários.



Brian W. Kernighan and Dennis M. Ritchie.

C: a linguagem de programação — padrão ANSI.

Campus, Rio de Janeiro, 1989.



K. N. King.

C Programming: A Modern Approach.

W. W. Norton, New York, 2 edition, 2008.

Capítulo 16: estruturas, uniões e enumerações.



Steve McConnell.

Code Complete: A Practical Handbook of Software Construction.

Microsoft Press, Redmond, WA, 2 edition, 2004.

Cap. 11 (nomes de variáveis), cap. 31 (layout e estilo) e cap. 32 (código autodocumentado).



Robert Mecklenburg.

Managing Projects with GNU Make.

O'Reilly Media, 3 edition, 2004.



Herbert Schildt.

C completo e total.

Makron Books, São Paulo, 3 edition, 1997.



Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.
AddressSanitizer: A fast address sanity checker.
In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.
Ferramenta usada no tutorial para detectar vazamentos e uso após free.



Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
Estruturas de dados usando C.
Pearson Makron Books, São Paulo, 1995.



The Linux Kernel Developers.
Linux kernel coding style.
Documentação do kernel Linux, `Documentation/process/coding-style.rst`.
Acesso em 2026.



The LLVM Project.
Clang-format style options.
Documentação do Clang.
Acesso em 2026.

-  Paul R. Wilson, Mark S. Johnstone, Michael Neely, and David Boles.
Dynamic storage allocation: A survey and critical review.
In International Workshop on Memory Management (IWMM), pages 1–116. Springer, 1995.
Panorama de estratégias de alocação dinâmica e fragmentação.
-  Nivio Ziviani.
Projeto de algoritmos: com implementações em Pascal e C.
Cengage Learning, São Paulo, 3 edition, 2011.